자료구조

01 주차

자료구조와 알고리즘 개요

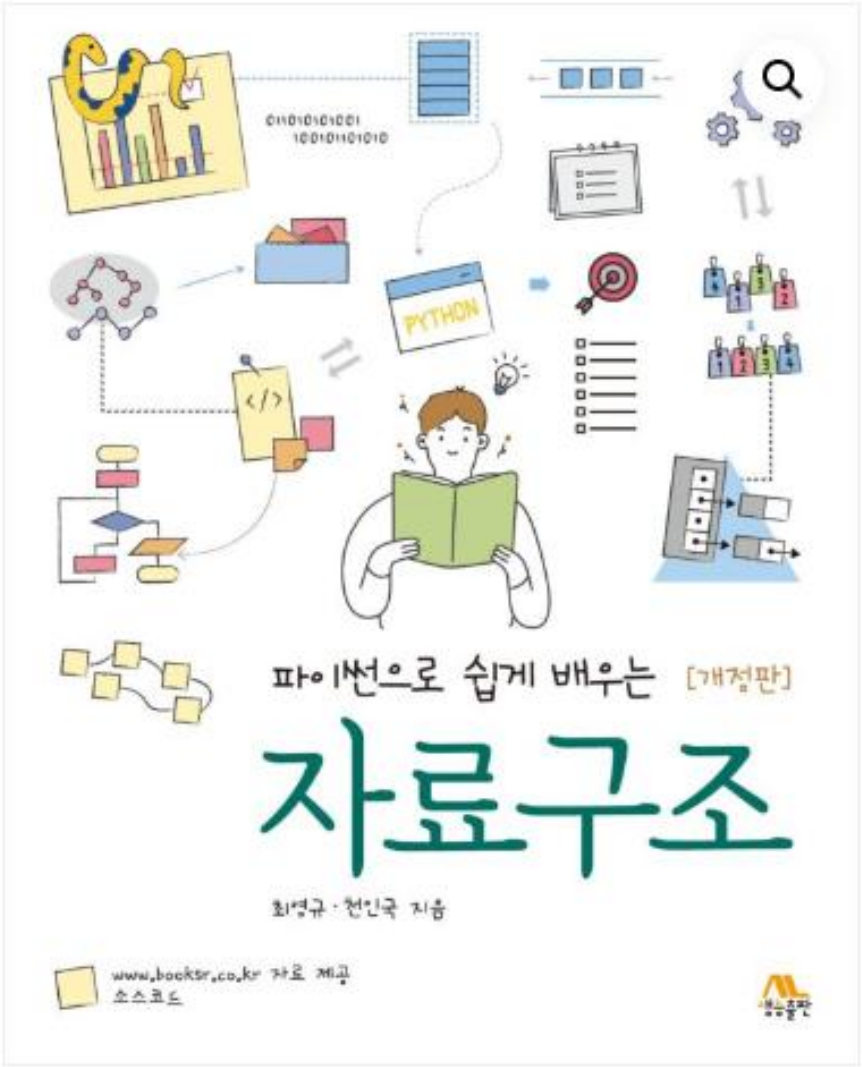


담당 교수

배 재 연

E-mail	jbae90@dongyang.ac.kr / spy_neo@naver.com
No.	010-5659-5709

강의 교재



파이썬으로 쉽게 배우는 자료구조 [개정판]

저자: 최영규, 천인국

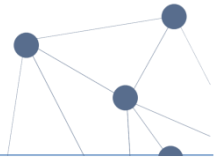
출판사: 생능 출판

평가 비율

출석 20%		중간 40% 필기		기말 40% 실기
수업시간 1시간 경과 후 무단 지각의 경우 결석 처리 출석률이 50%를 넘기지 못 한 경우 낙제 처리	+	8주차 2~7주차 강의 기반 중간평가	+	15주차 9~14주차 강의 기반 기말평가

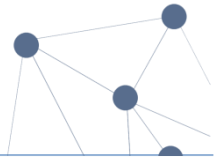
※ 모든 평가는 학교 규정 및 학업 성적 처리 지침에 따름

강의 일정



주차	날짜	주제
2	09/03	파이썬 리뷰 & 기본 자료형
3	09/10	리스트와 집합
4	09/17	스택
-	09/24	추석 휴강
5	10/01	큐, 덱, 우선순위 큐
6	10/08	연결 구조
7	10/15	정렬과 탐색 기초
8	10/22	중간고사

강의 일정



주차	날짜	주제
9	10/29	트리 기본
10	11/05	탐색 트리
11	11/12	다양한 트리 구조
12	11/19	힙 & 우선순위 큐
13	11/26	그래프 기본
14	12/03	그래프 심화 알고리즘
15	12/10	기말고사

1장. 자료구조와 알고리즘



1.1 자료구조와 알고리즘

1.2 추상 자료형

1.3 알고리즘의 성능 분석

1.4 반복 vs 순환 알고리즘

1.1 자료구조와 알고리즘

- 자료구조(Data Structure)란?
  - 컴퓨터에서 자료를 정리하고 조직화하는 다양한 구조
  - 생활 속의 다양한 사물과 관련된 자료구조의 예



배달할 선물 목록: 리스트(List)



접시나 물건 쌓기: 스택(Stack)



매표소 줄서기: 큐(Queue)

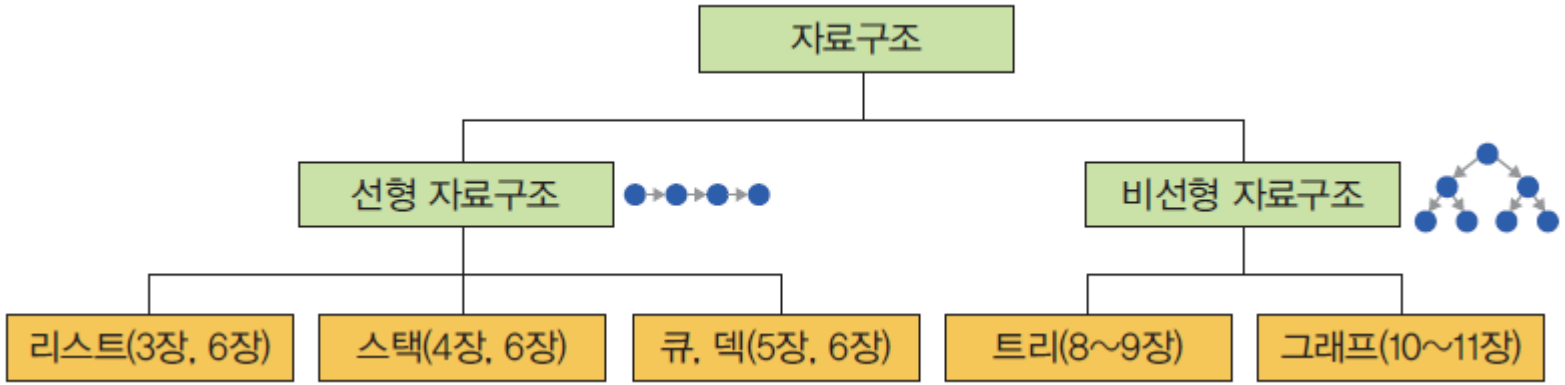


지하철 노선도: 그래프(Graph)



직장의 조직도: 트리(Tree)

자료구조의 분류



- 선형 자료구조
  - 자료를 순서적으로 나열하여 저장하는 창고
  - 접근 방법에 따라 다시 세분화: 리스트, 스택, 큐, 덱 등
- 비선형 자료구조
  - 복잡한 연결 관계의 자료 표현
    - 트리: 회사의 조직도나 컴퓨터의 폴더와 같은 계층 구조
    - 그래프: 가장 복잡한 연결 관계를 표현

알고리즘이란?

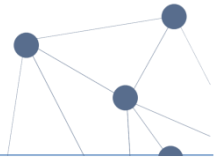
- 컴퓨터로 문제를 풀기 위한 단계적인 절차
  - 예) 사전에서 단어 찾기



- 프로그램 = 자료구조 + 알고리즘

  = 효율적인 문제 해결

알고리즘의 조건



- 입력: 0개 이상의 입력이 존재하여야 한다.
- 출력: 1개 이상의 출력이 존재하여야 한다.
- 명백성: 각 명령어의 의미는 모호하지 않고 명확해야 한다.
- 유한성: 한정된 수의 단계 후에는 반드시 종료되어야 한다.
- 유효성: 각 명령어들은 실행 가능한 연산이어야 한다.

알고리즘의 기술 방법

(1) 자연어

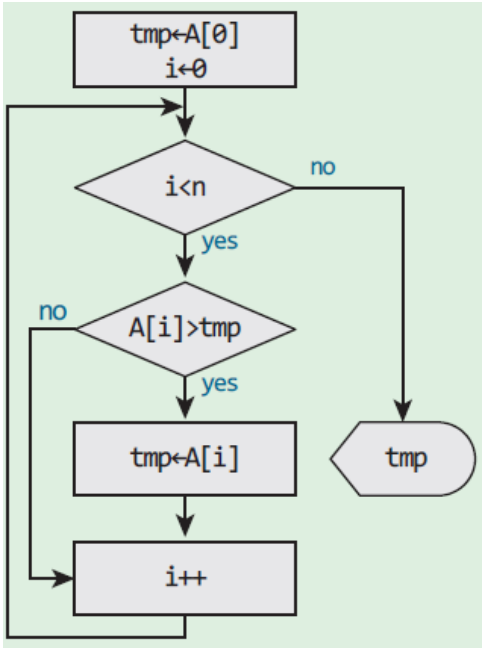
- 읽기 쉬움. 단어들을 정확하게 정의하지 않으면 의미 모호.

find_max(A)

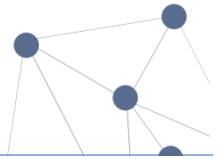
1. 배열 A의 첫 번째 요소를 변수 tmp에 복사한다.
2. 배열 A의 다음 요소들을 차례대로 tmp와 비교하여, 더 크면 그 값을 tmp로 복사한다.
3. 배열 A의 모든 요소를 비교했으면 tmp를 반환한다.

(2) 흐름도

- 직관적.
- 이해하기 쉬움.
- 복잡한 알고리즘→상당히 복잡!



알고리즘의 기술 방법



(3) 유사 코드

- 프로그램을 구현할 때의 여러 가지 문제들을 감출 수 있음
- 알고리즘의 핵심적인 내용에만 집중 가능

```
find_max(A)
tmp ← A[0]
for i←1 to size(A) do
        if tmp < A[i] then
                tmp ← A[i]
return tmp
```

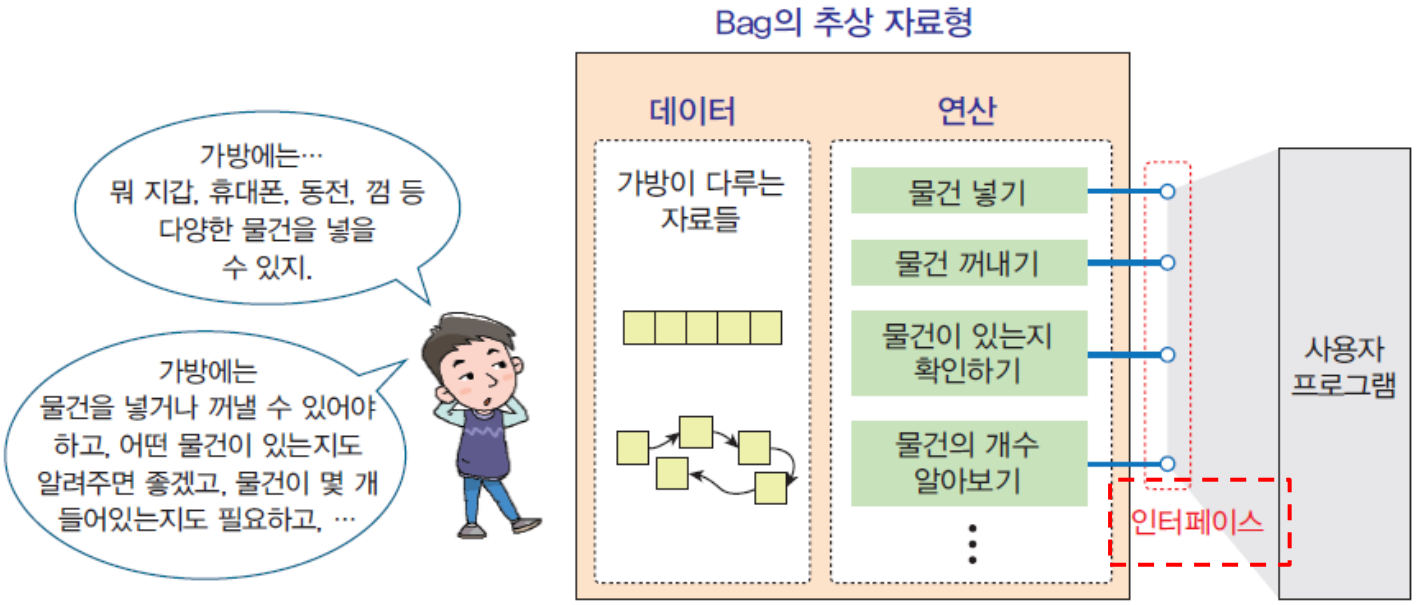
```
def find_max( A ):
    tmp = A[0]
    for item in A :
        if item > tmp :
            tmp = item
    return tmp
```

(4) 특정 언어

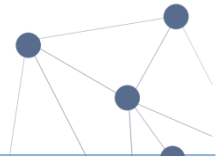
- 구현시의 사항들이 알고리즘의 핵심적인 내용들의 이해를 방해
- **파이썬**: C나 자바보다 훨씬 간결한 표현 가능
  - 바로 실행할 수 있음!

1.2 추상 자료형

- 프로그래머가 추상적으로 정의한 자료형
  - 어떤 자료를 다루고 이들에 대해 어떤 연산이 제공되는지를 기술
    - 데이터나 연산이 **무엇(what)**인가를 정의함
    - 데이터나 연산을 **어떻게(how)** 구현할 것인지는 정의하지 않음
  - 예: 가방(Bag)이 다루는 데이터와 연산들



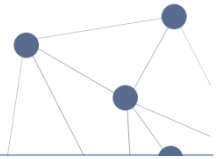
예) Bag의 추상 자료형



- 데이터
  - 중복된 항목을 허용하는 자료의 모임.
  - 항목들 사이에 순서는 없지만 서로 비교할 수는 있어야 함.

- 연산
  - insert(e): 가방에 항목 e를 넣는다.
  - remove(e): 가방에 e가 있는지 검사해 있으면 이 항목을 꺼낸다.
  - contains(e): e가 들어있으면 True를 없으면 False를 반환한다.
  - count(): 가방에 들어 있는 항목들의 수를 반환한다.

예) Bag 추상 자료형의 구현



- 파이썬 함수를 이용한 Bag 연산 구현 예

```
01  def contains(bag, e) :
02      return e in bag
03
04  def insert(bag, e) :
05      bag.append(e)
06
07  def remove(bag, e) :
08      bag.remove(e)
09
10  def count(bag):
11      return len(bag)
```

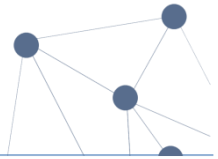
bag에 항목 e가 있는지 검사하는 함수. 파이썬의 in 연산자를 이용했는데, e가 bag에 있으면 True를 없으면 False를 반환함

bag에 새로운 항목 e를 넣는 함수. 파이썬 리스트의 append() 연산을 이용해 리스트의 맨 뒤에 e를 추가함

bag에서 항목 e를 삭제하는 함수. 파이썬 리스트의 remove() 연산을 이용해 구현함

bag에 들어 있는 항목의 수를 반환하는 함수. 파이썬 내장함수 len()을 이용함

예) Bag의 활용



- Bag을 이용한 자료 관리 예

```
01  myBag = []
02  insert(myBag, '휴대폰')
03  insert(myBag, '지갑')
04  insert(myBag, '손수건')
05  insert(myBag, '빗')
06  insert(myBag, '자료구조')
07  insert(myBag, '야구공')
08  print('가방속의 물건:', myBag)
09
10  insert(myBag, '빗')
11  remove(myBag, '손수건')
12  print('가방속의 물건:', myBag)
```

bag을 위한 새로운 배열을 만듦. 자료구조의 데이터를 저장하는 공간

새로운 bag인 myBag에 '휴대폰', '지갑', '손수건', '빗', '자료구조', '야구공'을 순서대로 삽입함

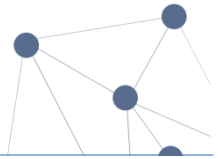
현재 myBag의 내용을 화면에 출력함

myBag에 '빗'을 추가로 삽입하고, '손수건'을 삭제함

변경된 myBag 내용을 화면에 출력

```
C:₩WINDOWS₩system32₩cmd.exe
내 가방속의 물건: ['휴대폰', '지갑', '손수건', '빗', '자료구조', '야구공']
내 가방속의 물건: ['휴대폰', '지갑', '빗', '자료구조', '야구공', '빗']
```

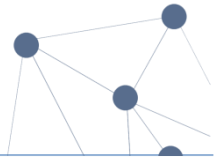
1.3 알고리즘의 성능 분석



- **실행 시간을 측정하는 방법**
  - 알고리즘의 실제 실행 시간을 측정하는 것
  - 알고리즘을 실제로 구현해야 함
  - 동일한 HW/SW 환경을 사용하여야 함

- **알고리즘의 복잡도를 분석하는 방법**
  - 직접 구현하지 않고서도 수행 시간을 분석
  - 알고리즘이 수행하는 연산의 횟수를 측정하여 비교
  - 연산의 횟수는 입력의 크기 n의 함수
    - **시간 복잡도 분석** : 수행 시간 분석
    - **공간 복잡도 분석** : 필요한 메모리 공간 분석

(1) 실행시간 측정



- 파이썬의 실행시간 측정 코드 예

```
01  import time
02
03  myBag = []
04  start = time.time()
05  insert(myBag, '축구공')
06  ...
07  end = time.time()
08  print("실행시간 = ", end-start)
```

시각 측정 함수를 사용하기 위해 time 모듈을 프로그램에 포함시킴

현재 시각을 start에 저장(알고리즘 처리 전)

실행시간을 측정하려는 코드(알고리즘)가 들어가는 부분

현재 시각을 end에 저장(종료 시각)

알고리즘의 전체 실행시간(end-start)

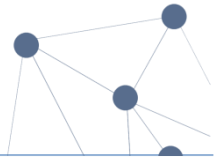
(2) 복잡도 분석

- 구현하지 않고 알고리즘의 효율성을 평가
  - 알고리즘의 연산 횟수를 대략적으로 계산
  - 복잡도 함수 T(n): 입력의 크기 n에 대한 알고리즘 연산 횟수

- 예: 1부터 n까지 합을 구하는 두 가지 알고리즘



복잡도 함수



- 알고리즘 1: T(n) = 2n+1

```
01  calc_sum1( n )
02        sum ← 0
03        for i ← 1 to n then
04           sum ← sum + i
05        return sum
```

← 연산자 1번 수행

반복 제어 연산은 무시

n번 반복되는 반복문 안에 있으므로 ← 연산자와 + 연산자가 각각 n번씩 수행

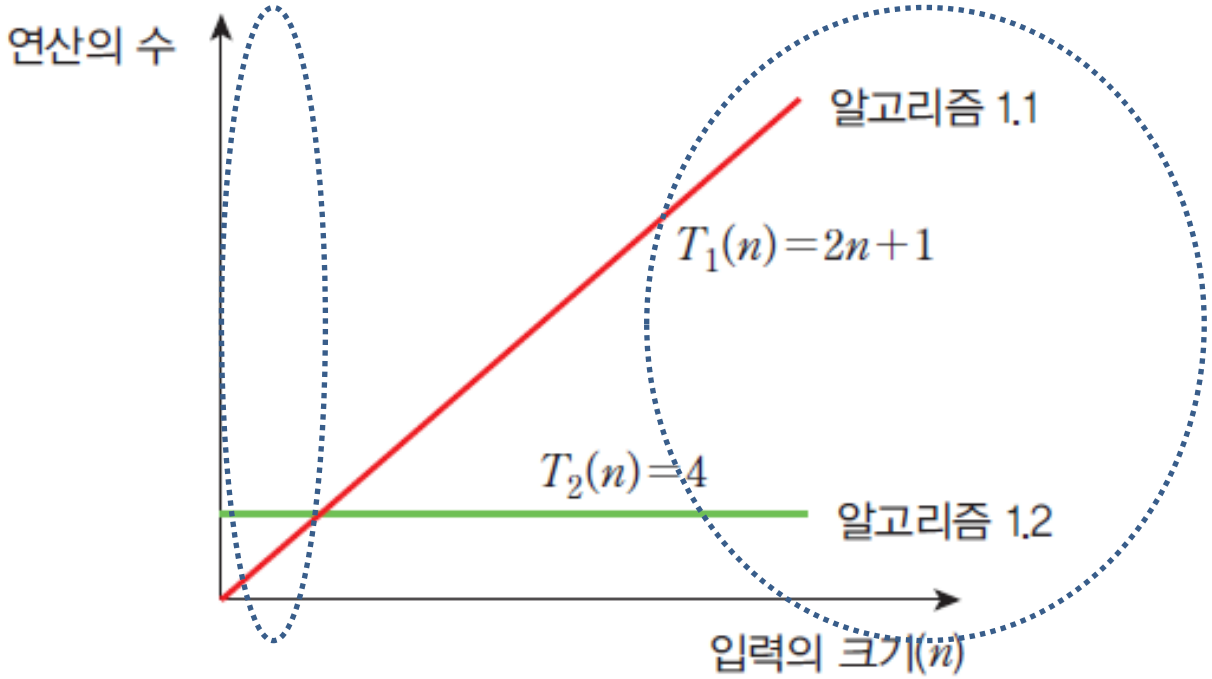
- 알고리즘 2: T(n) = 4

```
01  calc_sum2( n )
02        sum ← n * (n+1) / 2
03        return sum
```

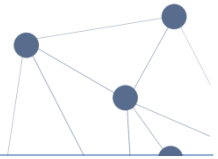
← 연산자와 *, +, / 연산자가 각각 1번씩 수행

알고리즘 비교

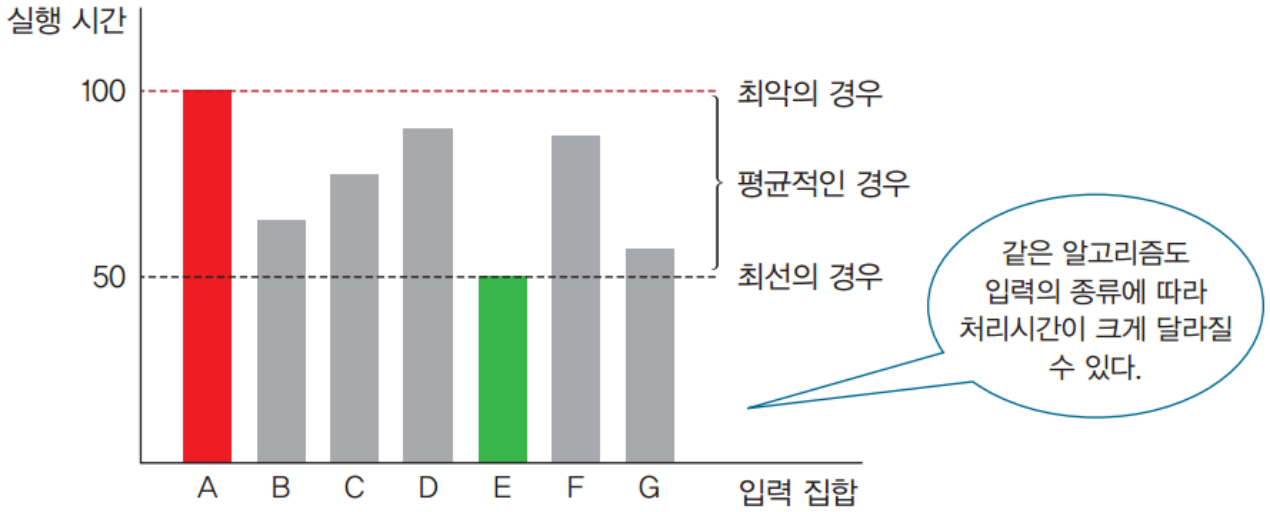
- 두 알고리즘의 성능 비교



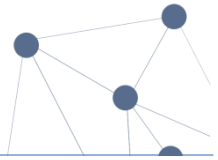
최선, 평균, 최악의 경우



- 실행시간은 입력 집합에 따라 다를 수 있음
  - 최선의 경우(best case): 수행 시간이 가장 빠른 경우
    - 의미가 없는 경우가 많다.
  - 평균의 경우(average case): 수행시간이 평균적인 경우
    - 계산하기가 상당히 어려움.
  - **최악의 경우(worst case)**: 수행 시간이 가장 늦은 경우
    - 가장 널리 사용됨. 계산하기 쉽고 응용에 따라서 중요한 의미를 가짐. (예) 비행기 관제업무, 게임, 로보틱스



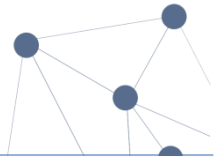
1.4 반복 vs 순환 알고리즘



- 같은 일을 되풀이하는 방법: 반복 / 순환
- **반복(iteration)**
  - for나 while 등의 반복문 이용
  - 보통은 수행속도가 빠름
  - 문제에 따라 프로그램 작성이 어려울 수 있음

- **순환(recursion)**
  - 알고리즘이나 함수가 수행 도중에 자기 자신을 다시 호출하여 문제를 해결하는 기법
  - 함수 호출의 오버헤드가 있음
  - 간결한 코딩이 가능

반복 vs 순환



반복

```python
def sum1(n):
    result = 0
    for i in range(1, n + 1):
        result += i
    return result
```

반복문이 n번 실행

```
sum1(4)

i=1 → result = 1
i=2 → result = 3
i=3 → result = 6
i=4 → result = 10
```

단순한 연산 반복

순환

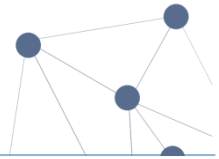
```python
def sum2(n):
    if n == 0:
        return 0
    return n + sum2(n - 1)
```

함수가 n번 자기 자신 호출

```
sum2(4)
 ↓
4 + sum2(3)
      ↓
   3 + sum2(2)
        ↓
     2 + sum2(1)
          ↓
       1 + sum2(0)
                ↓
                0
```

트리 탐색 / 하노이의 탑 등, 작은 문제의 반복

Summary



자료구조

- 선형
- 비선형

추상 자료형 (주로 클래스)

- 데이터 + 연산 포함
- 인터페이스 제공 (실제 동작 구현 필요)

알고리즘

- 자료/데이터에 대한 동작/연산 방식

알고리즘 기술 방식

- 자연어, 흐름도, 유사코드, 실제코드

알고리즘 성능 분석

- 실행 시간 : 직접 코드 구현
- 복잡도 : 알고리즘 연산 횟수 측정/비교
- 보통 "최악"의 경우 기준으로 판단/구현



감사합니다!